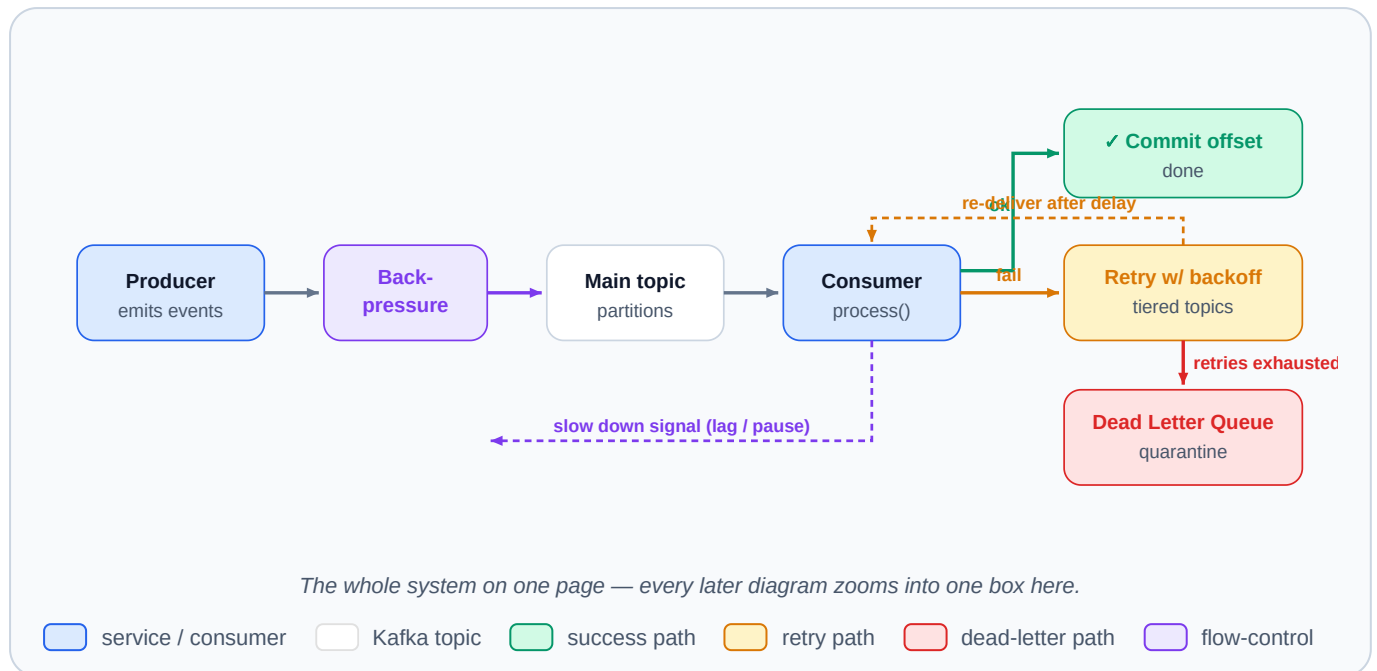


# Reliable Message Processing

A visual, flow-first guide to the four patterns that keep an event-driven pipeline alive under failure: **Retries**, **Dead Letter Queues**, **Backpressure**, and **Poison Pills**.



**Retries** — give a transient failure another chance, spaced out so you don't hammer a struggling dependency.

**DLQ** — a side channel where messages land once retries are exhausted, for inspection & replay.

**Backpressure** — let a slow consumer tell the upstream to ease off, instead of drowning.

**Poison pill** — a message that can *never* succeed; must be ejected before it blocks everything behind it.

## 1 Retries & Exponential Backoff

Most failures are **transient** — a timeout, a brief network blip, a dependency mid-deploy. A retry simply tries again. The art is *when* to retry, *how many times*, and *how to space attempts* so retries help instead of amplifying an outage.

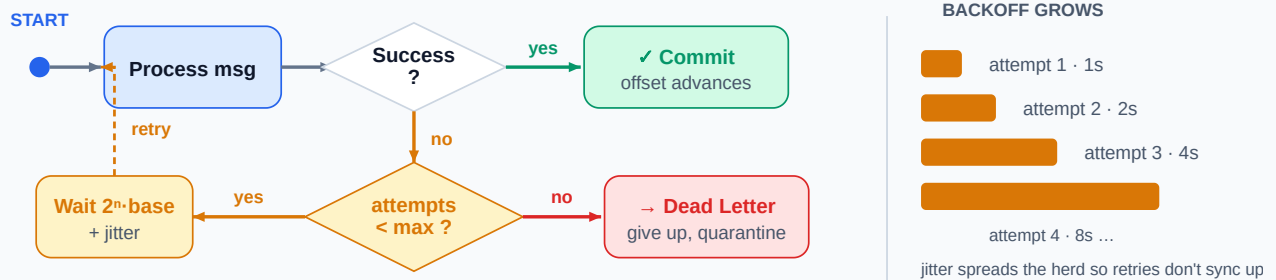


Fig 1 — The retry decision loop. Success commits; transient failure waits an exponentially growing delay and retries; exhausting the budget hands off to the DLQ.

**Retry these:** timeouts, 5xx, connection resets, throttling (429), optimistic-lock conflicts — anything likely to clear on its own.

**Don't retry these:** validation errors, 4xx, deserialization failures, null-pointer bugs. Re-running identical input just fails identically → send straight to DLQ.

**Why exponential + jitter?** Fixed, immediate retries turn one failure into a stampede against a dependency that's already down. Doubling the wait (1 → 2 → 4 → 8s) gives it room to recover; random *jitter* stops thousands of consumers retrying in lockstep (the "thundering herd").

**In-place vs. non-blocking:** retrying *inside* the poll loop blocks every message behind it (see §4). At scale, retries are done by publishing to separate *retry topics* — the next section.

## 2 Dead Letter Queues & Tiered Retry

A **Dead Letter Queue** is a separate topic where messages land after they've exhausted retries. It converts an invisible, blocking failure into a visible, inspectable, replayable one. The scalable pattern chains **retry topics of increasing delay** ahead of the DLQ so the live stream never blocks.

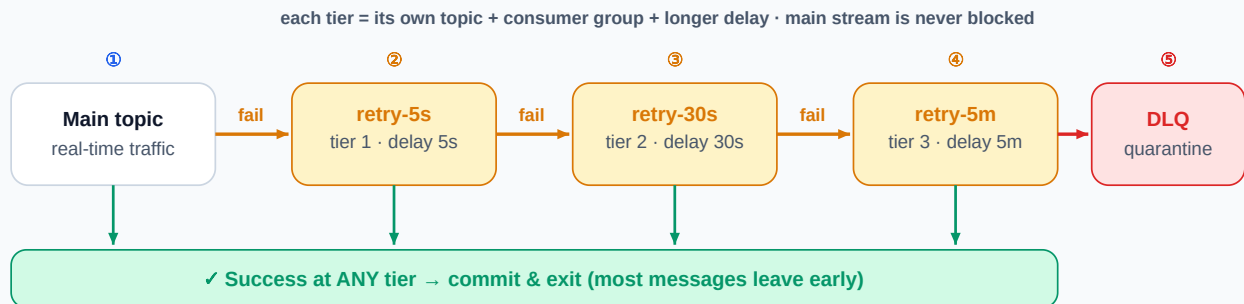


Fig 2 — Tiered retry topology. A failed message is **published forward** to the next delay tier and its original offset is committed, so healthy traffic keeps flowing. Only messages that fail every tier reach the DLQ.

### What you do with a DLQ

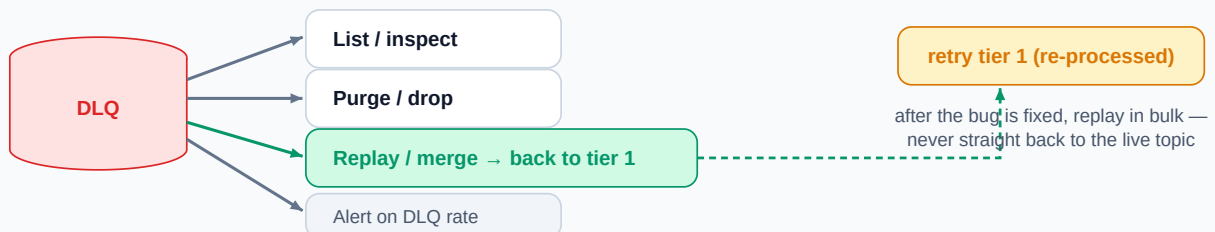


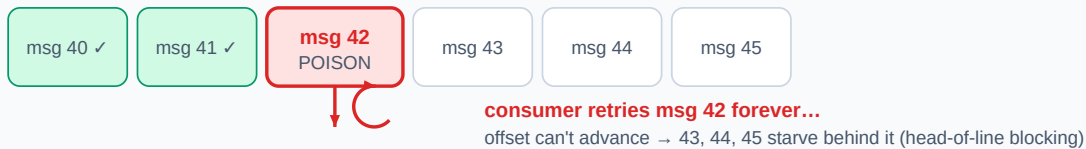
Fig 3 — A DLQ is an operational tool, not a graveyard: **list** to diagnose, **purge** the unrecoverable, **replay** the fixed, and **alert** when the inflow spikes.

**Golden rule:** replayed messages re-enter at the *first retry tier*, never the live topic — a bulk replay of 100k messages must never trample real-time traffic.

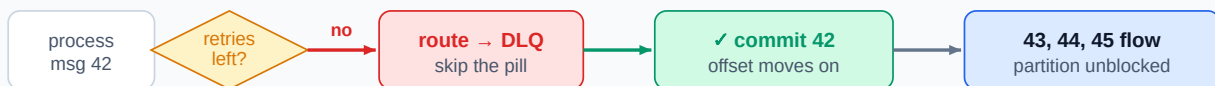
### 3 Poison Pills & Head-of-Line Blocking

A **poison pill** is a single message a consumer can *never* process — malformed JSON, an un-deserializable schema, a permanently invalid reference. Because a Kafka consumer reads a partition **strictly in order** and can't commit past an un-processed offset, one poison pill freezes every message behind it.

✗ BEFORE — the partition is stuck



✓ AFTER — detect, eject, keep moving



Preserve a

**correlation / trace id** and the failure reason on the DLQ record so the pill can be diagnosed and, once fixed, replayed.

*Fig 4 — The cure for a poison pill is a bounded retry budget that ends in the DLQ. Eject the one bad message so the offset can advance and the rest of the partition drains.*

**Detect it:** a message whose retry count keeps maxing out, a deserialization exception, or the same offset reprocessed repeatedly.

**Contain it:** wrap processing in a try/catch that, on non-retryable errors, routes to the DLQ and commits — never let one record halt the loop.

## 4 Backpressure & Flow Control

**Backpressure** is what happens when work arrives faster than it can be processed. Without a way to push back, an in-memory buffer grows without bound until the consumer runs out of memory. With backpressure, the slow stage signals upstream to **slow down** — trading latency for survival.

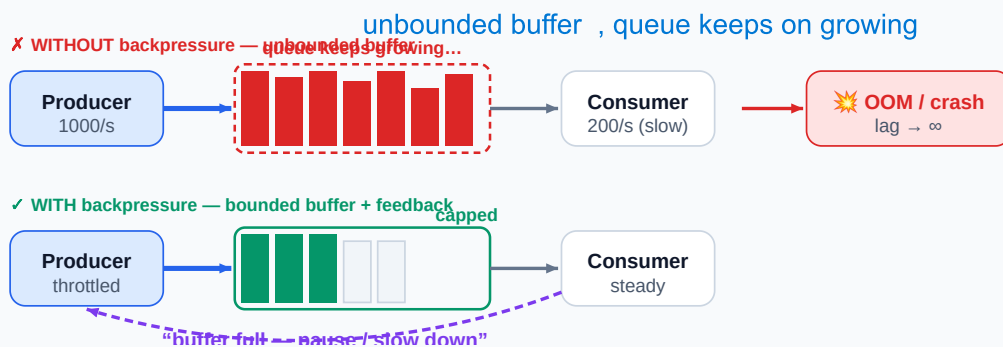


Fig 5 — Same mismatch, two outcomes. An unbounded buffer converts a speed gap into a crash; a bounded buffer plus an upstream signal converts it into graceful slowdown.

### How the signal is actually sent

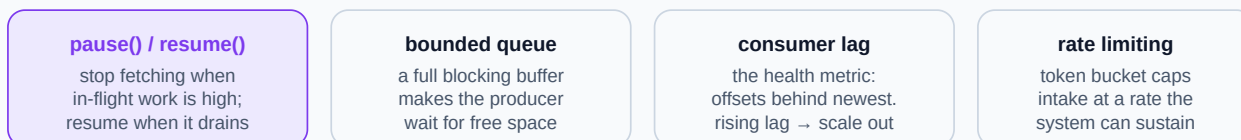


Fig 6 — Four everyday backpressure levers. Kafka's pull model is itself a form of backpressure: consumers fetch only when ready, so a slow consumer simply polls less — lag rises but nothing overflows.

**Backpressure vs. buffering:** a buffer *absorbs* a short burst; backpressure *regulates* a sustained overload. You need both — a buffer for spikes, and a signal for when the buffer can't keep up.



# How They Fit Together & Quick Reference

The four patterns are layers of one defense: backpressure protects the intake, retries absorb transient failures, poison-pill handling ejects the incurable, and the DLQ catches whatever survives — all so the main stream keeps flowing.

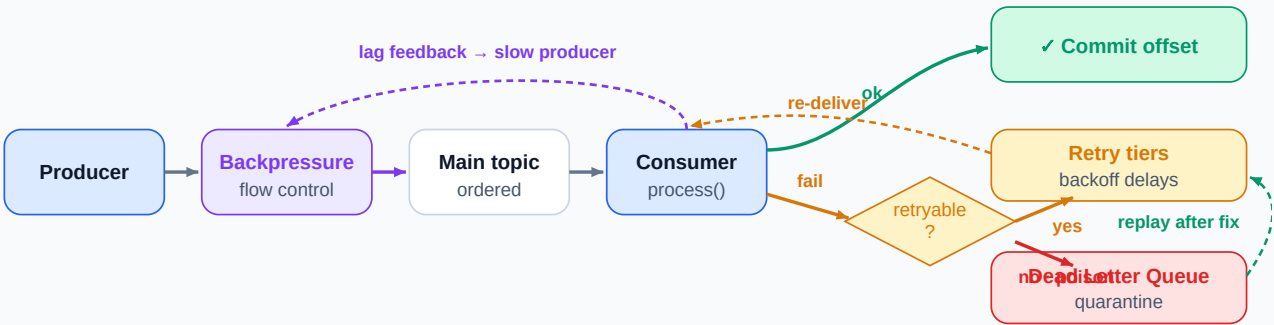


Fig 7 — One pipeline, four safeguards. Follow any failing message: it's retried with backoff, checked for poison, and only quarantined in the DLQ once nothing else can save it.

| Pattern      | Problem it solves                             | Key signal / trigger                         | Typical knobs                                      |
|--------------|-----------------------------------------------|----------------------------------------------|----------------------------------------------------|
| Retries      | Transient failure fails a good message        | retryable exception (timeout, 5xx, 429)      | max attempts, base delay, multiplier, jitter       |
| DLQ          | Failures need visibility & recovery, not loss | retry budget exhausted                       | DLQ topic, retention, list/purge/replay tooling    |
| Poison pill  | One bad message blocks the whole partition    | non-retryable / repeated same-offset failure | error classification, skip-to-DLQ, correlation id  |
| Backpressure | Producer outruns consumer → overflow          | rising consumer lag / full buffer            | bounded queue, pause/resume, rate limit, autoscale |

**Do** — classify errors (retryable vs. not), cap retries, add jitter, keep the DLQ inspectable, alert on DLQ inflow, make consumers idempotent (at-least-once means duplicates).

**Avoid** — infinite in-loop retries, retrying non-retryable errors, unbounded buffers, replaying a DLQ straight onto the live topic, dropping failures silently.